Computación Paralela y Distribuída

2026-II

José Fiestas PhD

October 15, 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Unidad 5: Análisis y programación de algoritmos paralelos

Objetivos:

1. Naturalmente (vergonzosamente) algoritmos paralelos
2. Ecuaciones diferenciales parciales
3. Algoritmos de ordenamiento en paralelo
4. Algoritmos de búsqueda en paralelo
5. Algoritmos de grafos en paralelo

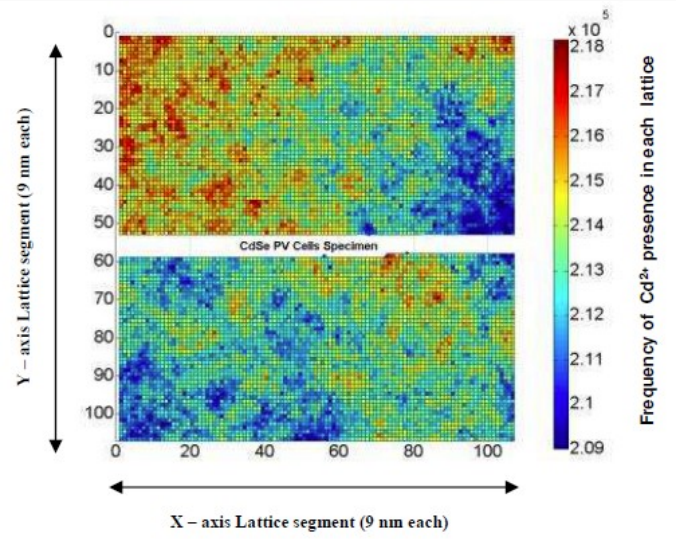
Ecuaciones Diferenciales Parciales

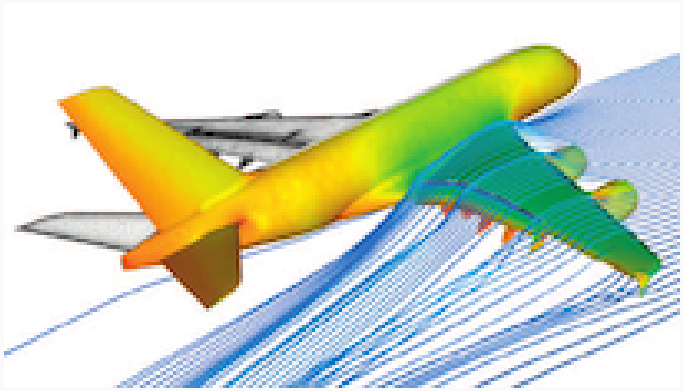
Ecuaciones Diferenciales Parciales

Miden cambios en funciones contínuas en modelos de sistemas en el campo de la Física, Dinámica de Fluidos, Biología, Química, y su evolución temporal, por lo que son relevantes en todas las ciencias, ingeniería e industria. E.g.

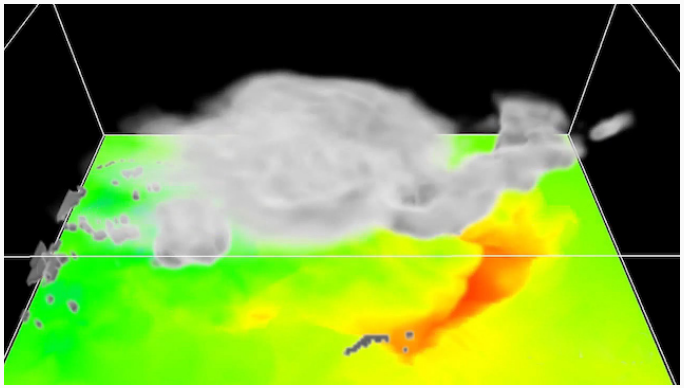
$$\frac{\partial u}{\partial x}(x,y) + \frac{\partial u}{\partial y}(x,y) = 0 \tag{1}$$

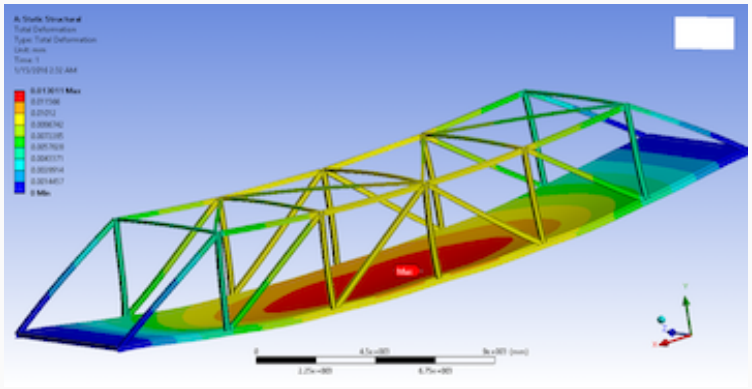
Con la solución $u = f(x,y)$
e.g. ecuaciones de calor, de difusión, de onda



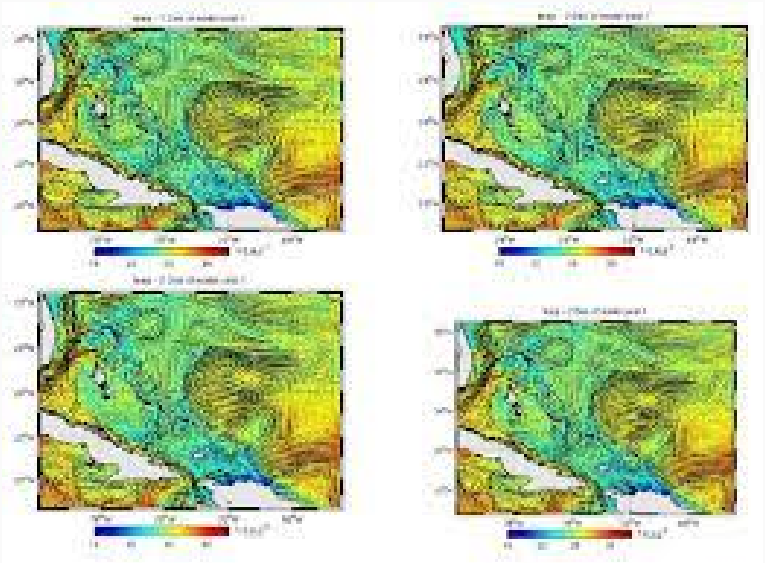


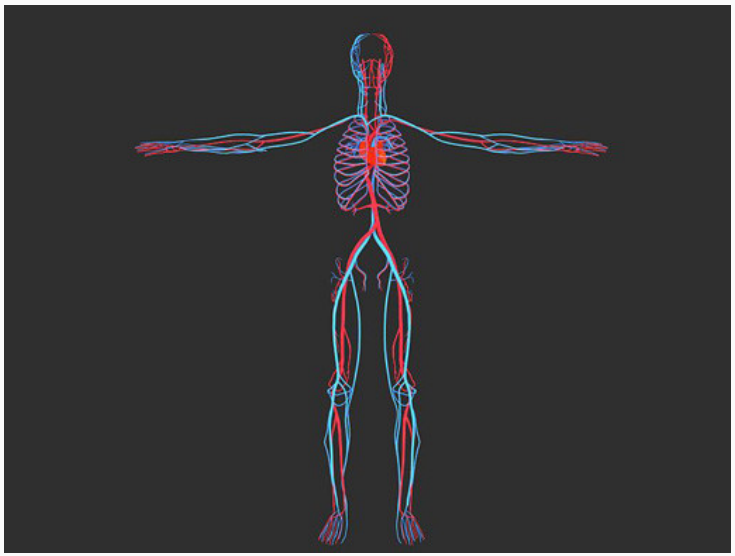
Ecuaciones Diferenciales Parciales





$$\frac{\partial u}{\partial x}(x,y) + \frac{\partial u}{\partial y}(x,y) = 0 \tag{2}$$





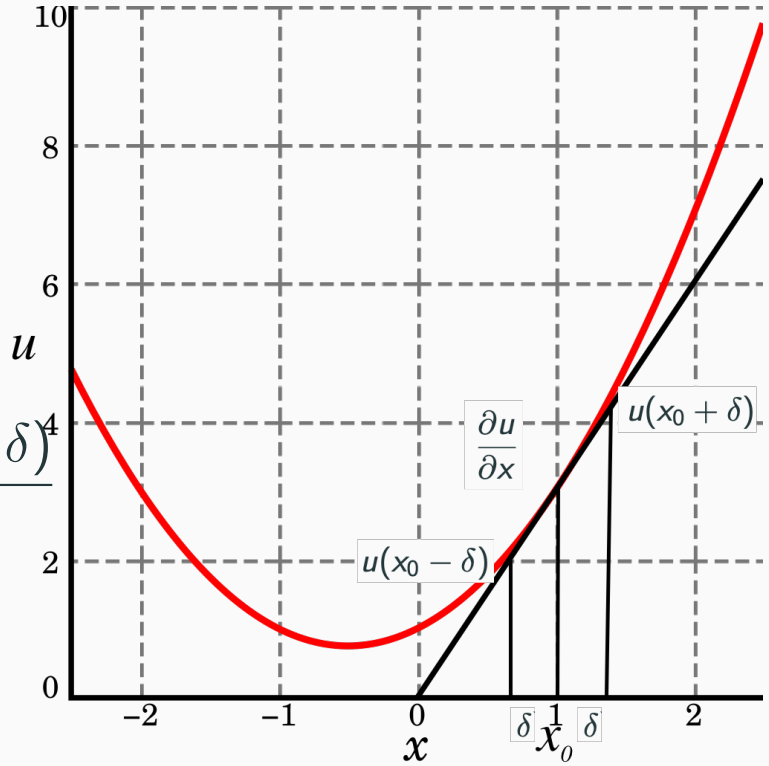
Aplicación de diferencias finitas en cálculo de derivadas

$$\frac{\partial u}{\partial x} = \frac{u(x_0+\delta) - u(x_0-\delta)}{2\delta}$$

o también

$$\frac{\partial^2 u}{\partial x^2} = \frac{u(x_0+\delta) - 2u(x_0) + u(x_0-\delta)}{\delta^2}$$

Esta discretización ayuda
a obtener el valor de u(x) según
la serie de Taylor, evaluada en x_0

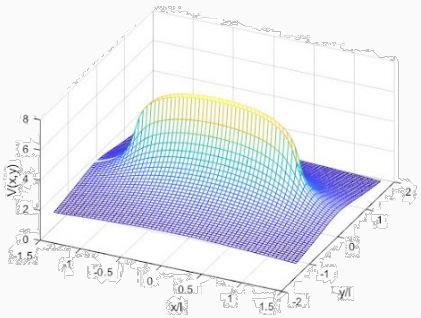


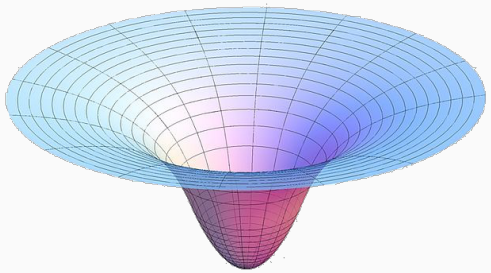
$$u(x_0) + \frac{u'(x_0)}{1!}(x-x_0) + \frac{u''(x_0)}{2!}(x-x_0)^2 + \frac{u'''(x_0)}{3!}(x-x_0)^3 + \ldots$$

Ecuaciones Diferenciales: tipos de EDP

Ecuaciones elípticas (Laplace, Poisson)

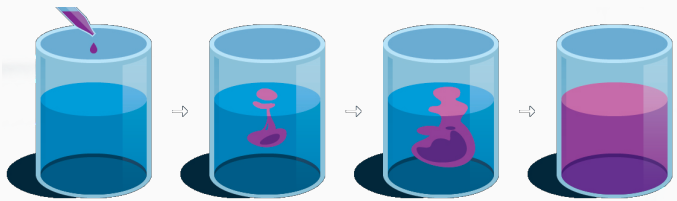
$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = S(x, y)$$

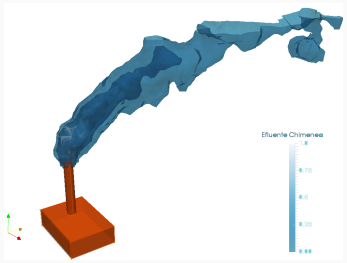




Ecuaciones parabólicas (ecuaciones de calor)

$$\frac{\partial u}{\partial t} = -\frac{\partial}{\partial x}(\frac{\partial u}{\partial x})$$

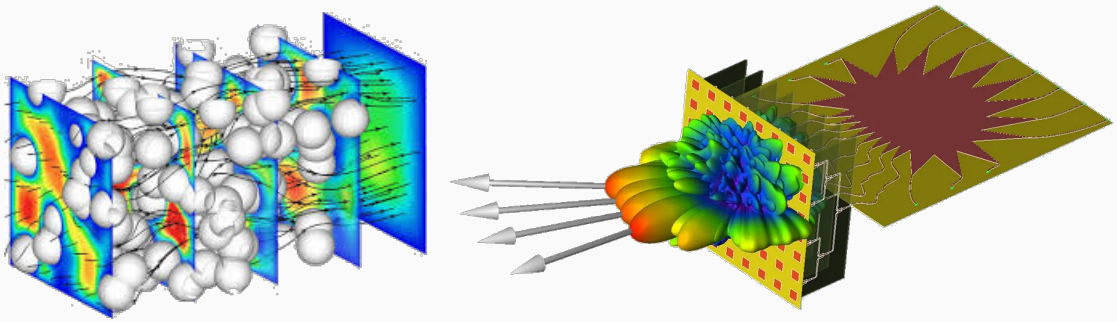


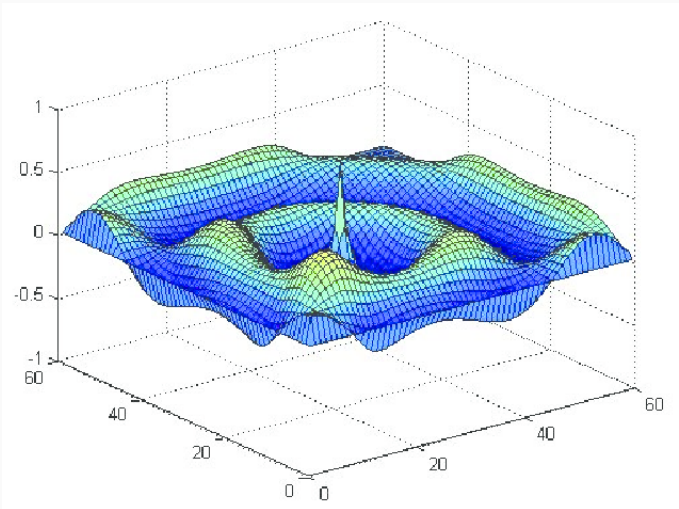


Ecuaciones Diferenciales: tipos de EDP

Ecuaciones hiperbólicas (ecuación de onda)

$$\frac{\partial^2 u}{\partial t^2} = -c^2 \frac{\partial^2 u}{\partial x^2}$$





Ecuacion de Laplace (elíptica)

Es un modelo matemático de flujo de calor y viscozidad. **u** es la variable dependiente (e.g. temperatura), **x-y** las coordenadas espaciales

$$\frac{\partial^2 u}{\partial x^2}(x,y) + \frac{\partial^2 u}{\partial y^2}(x,y) = 0$$

En un grid regular (i.e. dx , dy constantes), se puede substituir la ecuación finita diferencial de primer orden para una derivada parcial

$$\frac{\partial u}{\partial x} = \frac{u_{i+1,j} - u_{i,j}}{\Delta x} + O(\Delta x)$$

$$\frac{\partial u}{\partial y} = \frac{u_{i,j+1} - u_{i,j}}{\Delta y} + O(\Delta y)$$

Ecuacion de Laplace

La ecuación finita diferencial de segundo orden

$$\frac{\partial^2 u}{\partial x^2} = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{(\Delta x)^2} + O((\Delta x)^2)$$

$$\frac{\partial^2 u}{\partial y^2} = \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{(\Delta y)^2} + O((\Delta y)^2)$$

Substituyendo en la ecuación de Laplace, tenemos

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{(\Delta x)^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{(\Delta y)^2} + O((\Delta y)^2) + O((\Delta x)^2)$$

donde se puede descartar el término de error y asumir que Δx y Δy tienen la misma longitud

Ecuacion de Laplace: Método de las diferencias finitas

Al resolver la ecuación para $u_{i,j}^{n+1}$ y substrayendo $u_{i,j}^{n}$ de ambos lados, se obtiene

$$\Delta u_{i,j}^{n+1} = \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}}{4} - u_{i,j}^{n}$$

es decir,

$$u_{i,j}^{n+1} = u_{i,j}^{n} + \Delta u_{i,j}^{n+1}$$

donde se puede descartar el término de error y asumir que Δx y Δy tienen la misma longitud

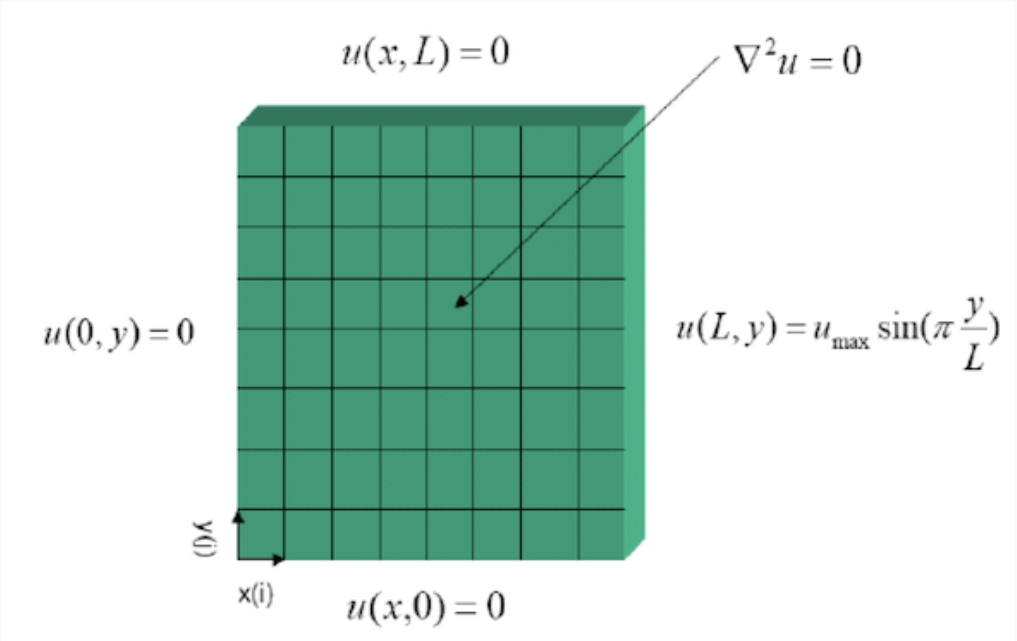
En código:

```
for(i=1;i<=NR;i++)
    for(j=1;j<=NC;j++)
        u[i,j]= 0.25*(uold[i-1,j] + uold[i+1,j] +uold[i,j-1]+uold[i,j+1]);
```

Ecuacion de Laplace: Condiciones de frontera

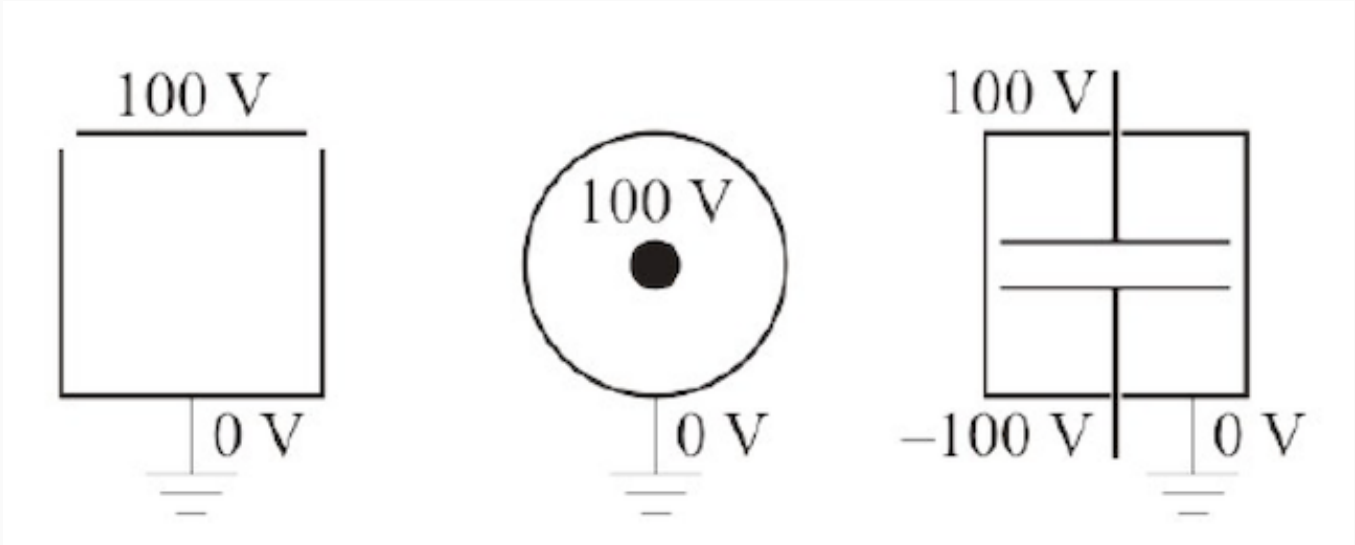
Inicialmente,
las condiciones de frontera
son:

- $0 \leq x \leq L$
- $0 \leq y \leq L$
- $u(0, y) = 0$
- $u(L, y) = u_{max} sin(\pi y / L)$
- $u(x, 0) = 0$
- $u(x, L) = 0$
- $u^0_{i,j} = 0$



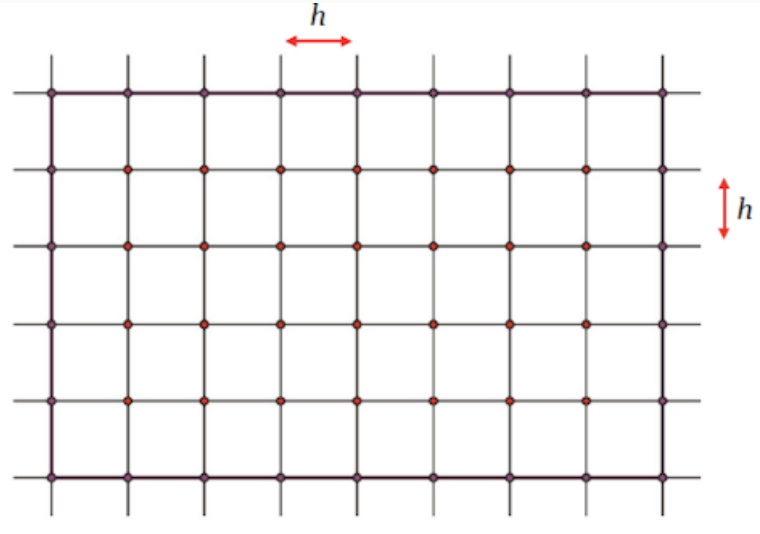
Ecuacion de Laplace: Condiciones de frontera

Pueden ser dadas por voltajes (pueden ser internos o externos)



Ecuacion de Laplace

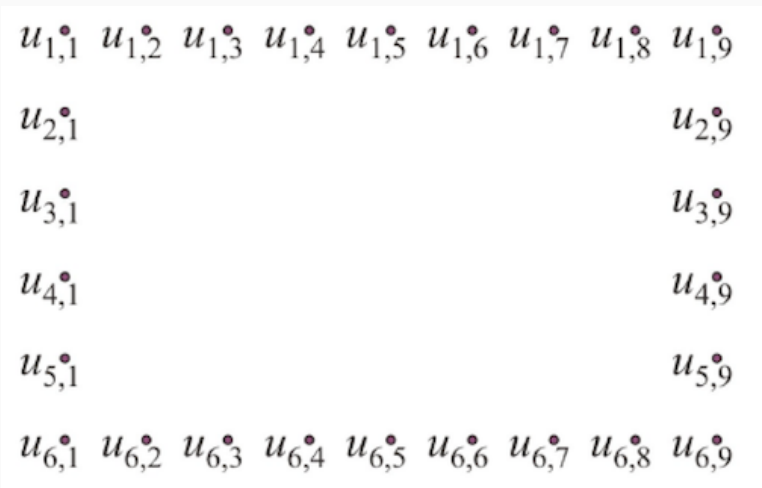
Considere la matriz u_{ij} sobre una malla de $n_x \times n_y$ puntos y distancia constante h

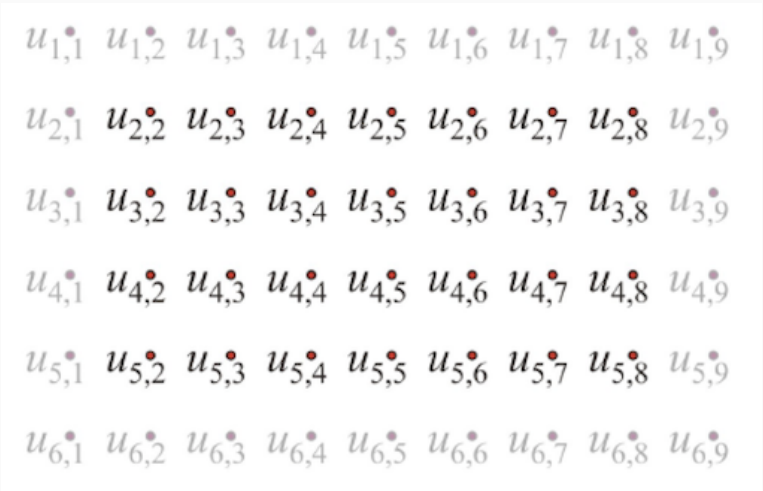


$$
\begin{array}{ccccccccc}
u_{1,1} & u_{1,2} & u_{1,3} & u_{1,4} & u_{1,5} & u_{1,6} & u_{1,7} & u_{1,8} & u_{1,9} \\
u_{2,1} & u_{2,2} & u_{2,3} & u_{2,4} & u_{2,5} & u_{2,6} & u_{2,7} & u_{2,8} & u_{2,9} \\
u_{3,1} & u_{3,2} & u_{3,3} & u_{3,4} & u_{3,5} & u_{3,6} & u_{3,7} & u_{3,8} & u_{3,9} \\
u_{4,1} & u_{4,2} & u_{4,3} & u_{4,4} & u_{4,5} & u_{4,6} & u_{4,7} & u_{4,8} & u_{4,9} \\
u_{5,1} & u_{5,2} & u_{5,3} & u_{5,4} & u_{5,5} & u_{5,6} & u_{5,7} & u_{5,8} & u_{5,9} \\
u_{6,1} & u_{6,2} & u_{6,3} & u_{6,4} & u_{6,5} & u_{6,6} & u_{6,7} & u_{6,8} & u_{6,9}
\end{array}
$$

Ecuacion de Laplace

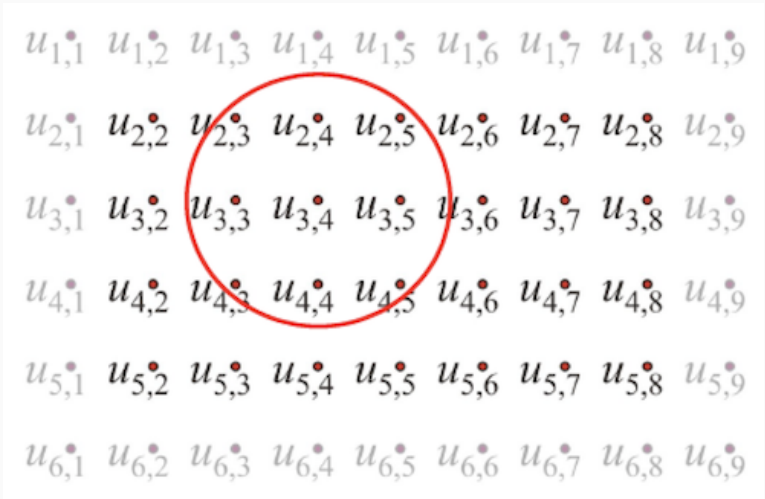
Representación de condiciones de frontera y valores intermedios

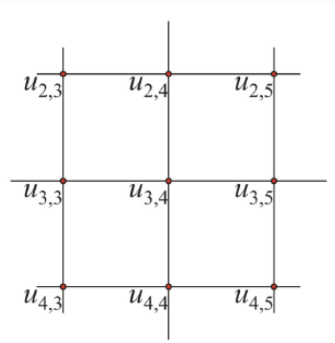




Ecuacion de Laplace

El cálculo en esta región,





Se implementa como $u(x,y) = \frac{u(x-h,y)+u(x+h,y)+u(x,y-h)+u(x,y+h)}{4}$

Con $x = x_3$, $y = y_4$,

$$-4u(x_3,y_4)+u(x_3-h,y_4)+u(x_3+h,y_4)+u(x_3,y_4-h)+u(x_3,y_4+h)=0$$

Y si hacemos $x_3 - h = x_2$, $x_3 + h = x_4$, $y_4 - h = y_3$, $y_4 + h = y_5$

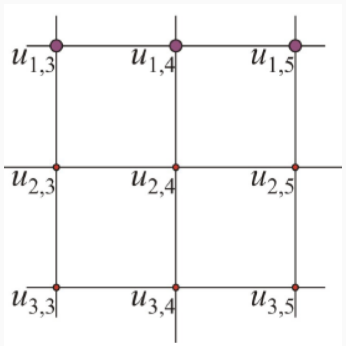
$$-4u(x_3,y_4)+u(x_2,y_4)+u(x_4,y_4)+u(x_3,y_3)+u(x_3,y_5)=0$$

$$-4u_{3,4}+u_{2,4}+u_{4,4}+u_{3,3}+u_{3,5}=0$$

Ecuacion de Laplace

Si uno de los valores adyacentes está en la frontera, entonces es un valor conocido





$-4u_{2,4} + u_{1,4} + u_{3,4} + u_{2,3} + u_{2,5} = 0$, o bien

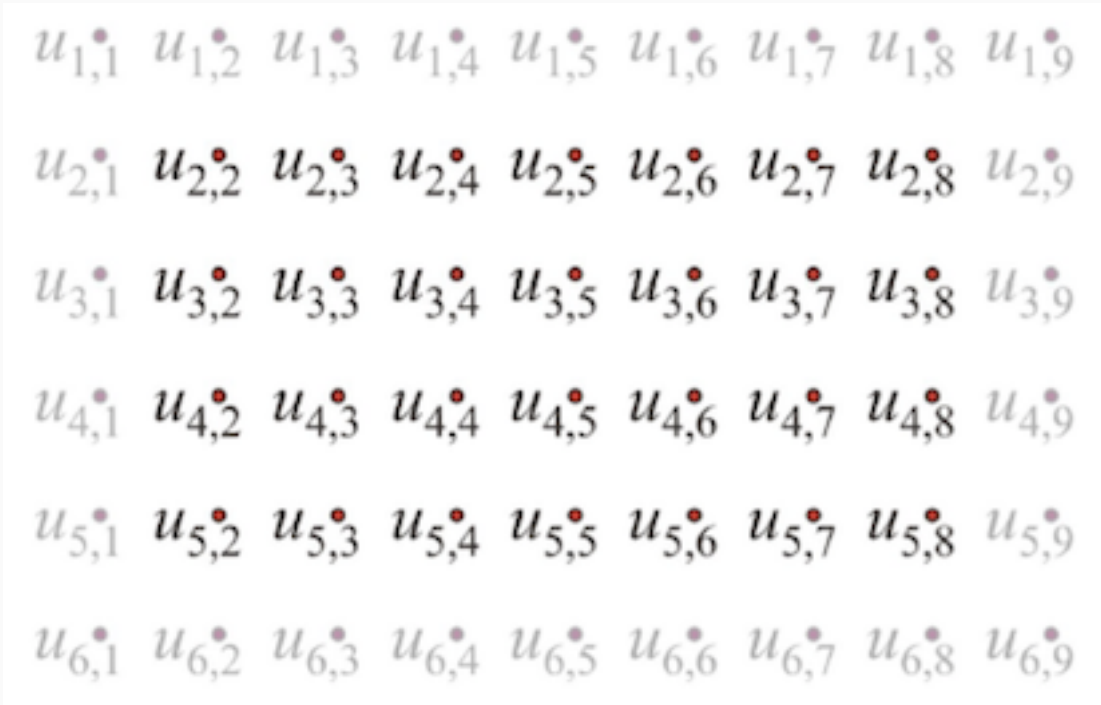
$-4u_{2,4} + u_{3,4} + u_{2,3} + u_{2,5} = +u_{1,4}$

Ecuacion de Laplace 2D

Es decir, para cada punto interior se puede escribir una ecuación

$$-4u_{i_x,i_y} + u_{i_x-1,i_y} + u_{i_x+1,i_y} + u_{i_x,i_y-1} + u_{i_x,i_y+1} = 0$$

Lo que da un sistema de n ecuaciones con n incógnitas



Laplace en 1 dimensión, un solo paso temporal

```
// Iteracion (metodo de Jacobi)
    while (error > tol && iter < max_iter) {
        u_old = u;
        error = 0.0;

        for (int i = 1; i < N - 1; ++i) {
            u[i] = 0.5 * (u_old[i - 1] + u_old[i + 1]);
            error = std::max(error, std::abs(u[i] - u_old[i]));
        }

        iter++;
    }
```

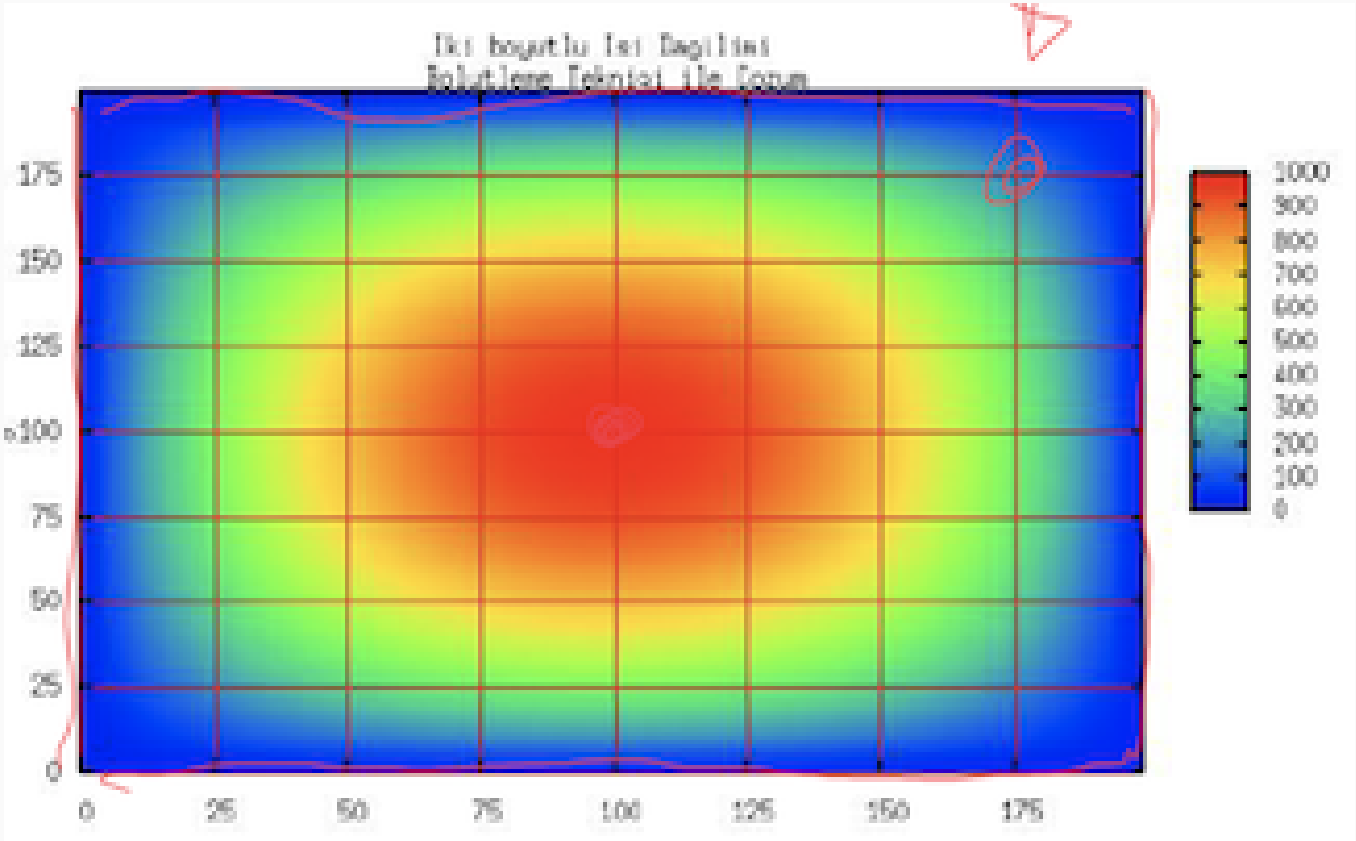
Laplace en 1 dimensión, con MPI

```
// Iteracion (metodo de Jacobi)
MPI_Scatter(sendbuf.data(), local_N, MPI_DOUBLE,
                &local_u[1], local_N, MPI_DOUBLE, 0, MPI_COMM_WORLD);

while (global_error >tol && iter < max_iter) {
   local_old = local_u;
// Comunicacion con vecino izquierdo
if (rank > 0) {
  MPI_Send(&local_old[1], 1, MPI_DOUBLE, rank-1, 0, MPI_COMM_WORLD);
  MPI_Recv(&local_old[0], 1, MPI_DOUBLE, rank-1, 1, MPI_COMM_WORLD, ...); }
// Comunicacion con vecino derecho
if (rank < size - 1) {
  MPI_Send(&local_old[local_N],1,MPI_DOUBLE,rank+1,1,MPI_COMM_WORLD);
  MPI_Recv(&local_old[local_N+1],1,MPI_DOUBLE,rank+1,0,MPI_COMM_WORLD,..);}
 double local_error = 0.0;
 for (int i = 1; i <= local_N; ++i) {
   if ((rank==0 && i==1) || (rank==size-1 && i==local_N)) continue;
        local_u[i] = 0.5 * (local_old[i - 1] + local_old[i + 1]);
 }
MPI_Allreduce(&local_error, &global_error, 1, MPI_DOUBLE, MPI_MAX, ...);
 iter++;
 }
MPI_Gather(&local_u[1], local_N, MPI_DOUBLE,
               global_u.data(), local_N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
 }
```

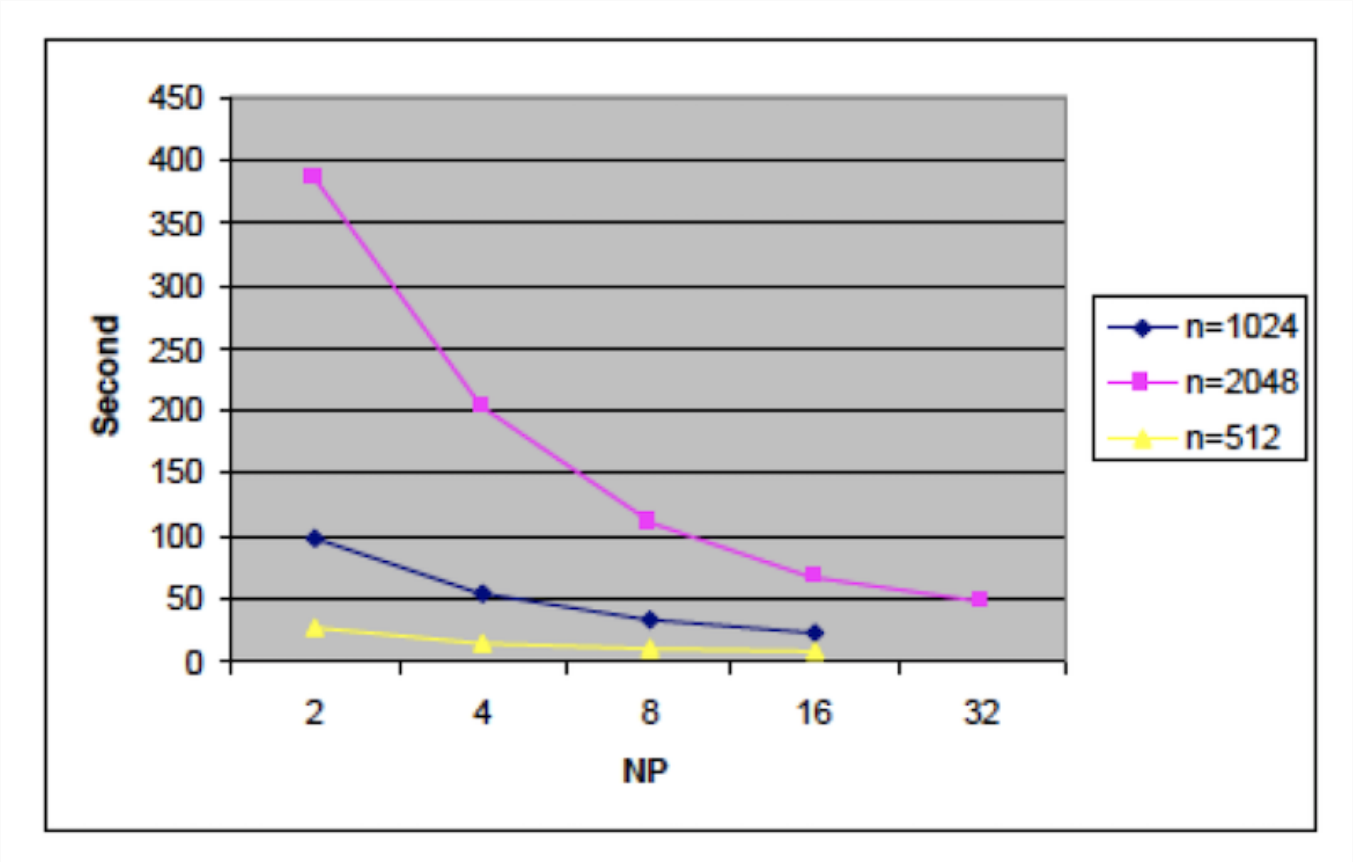
Ecuacion de Laplace 2D

Resultados



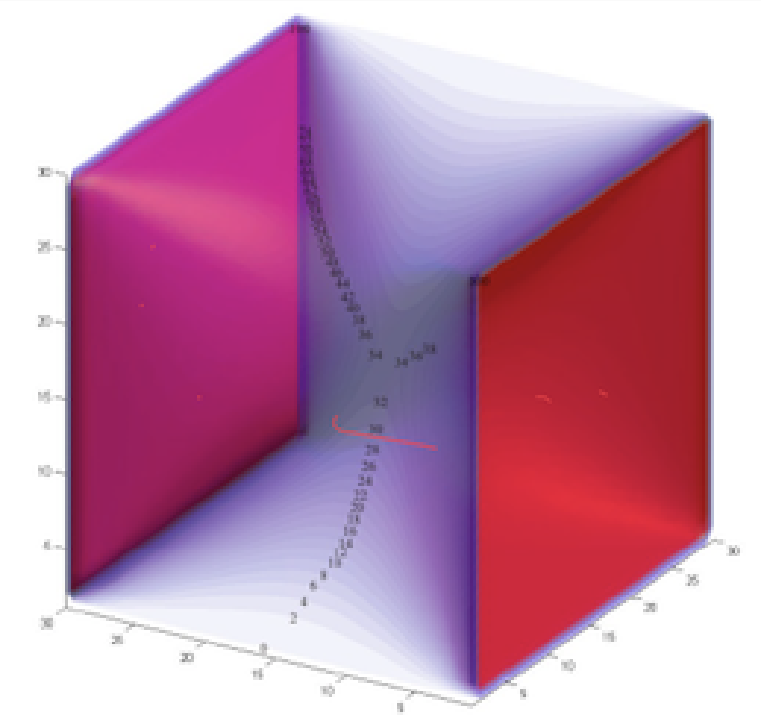
Ecuacion de Laplace 2D

Resultados (NP es número de procesos)



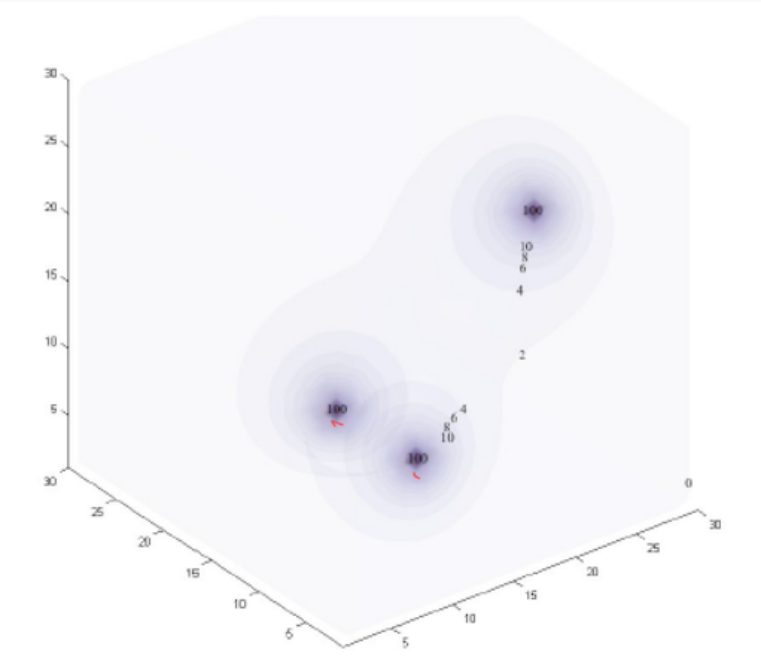
Ecuacion de Laplace: Condiciones de frontera

Temperaturas de frontera en 3D:
100° en lados opuestos y
0° en los otros dos



Ecuacion de Laplace: Condiciones de frontera

Temperaturas de frontera en 3D:
3 fuentes de calor
con paredes de 0°



Ecuación de Poisson

Soluciones de funciones de potenciales (gravedad, electróstatica), describen el estado de equilibrio en procesos de transmisión de energía La función u(x,y) es independiente del tiempo (i.e. potencial). S(x,y) representa la fuente (masa, carga o fuente de calor). Ya que son independientes en el tiempo, se les llama de tipo elíptico.

$$\nabla_u^2 = \frac{\partial u^2}{\partial x^2} + \frac{\partial u^2}{\partial y^2} = S(x, y)$$

Las soluciones se especifican a las fronteras del dominio (de 1,2...,N dimensiones):

- Condición de frontera de Dirichtet u(x,y)=f(x,y)
- Condición de frontera de Newman $\frac{\partial u}{\partial n} g(x, y)$

Ecuación de Poisson

Para la solución de **diferencia finita**, se define una malla de intervalos constantes ($h_x = h_y$) que cubren el dominio.
El número de puntos de malla es $N_x \cdot N_y$. El campo es $u(x_i, y_j) = u_{i,j}$, y la fuente es $S(x_i, y_j) = S_{i,j}$
Usando la fórmula de derivada parcial , la ecuación diferencial parcial es

$$\frac{1}{h^2}(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}) = S_{i,j}$$

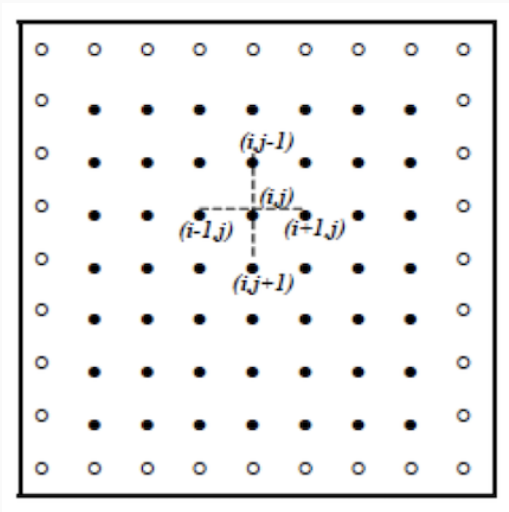
que forman un sistema lineal de ecuaciones, que puede ser resuelto para $u_{i,j}$
El número de incógnitas en este sistema es $N_x \cdot N_y$ en 2D. En 3D para una malla de 1000^3 pueden llegar a ser 10^9 ecuaciones

Ecuación de Poisson: solución numérica iterativa

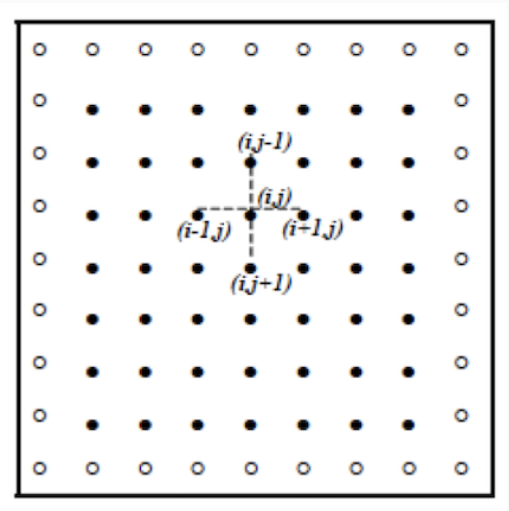
La solución
puede obtenerse en una iteración. Partiendo del valor inicial $u_{i,j}$ de los valores del dominio, el valor del campo en el punto de malla i y j se resuelve partiendo de los valores de los puntos vecinos. El índice k se refiere a la iteración

$$u_{i,j}^{k+1} = \frac{1}{4}(u_{i+1,j}^{k} + u_{i-1,j}^{k} + u_{i,j+1}^{k} + u_{i,j-1}^{k} - h^2 S_{i,j})$$

El **método de Jacobi** guarda los valores estimados en cada iteración y los aplica en la siguiente iteración



Ecuación de Poisson: solución numérica iterativa



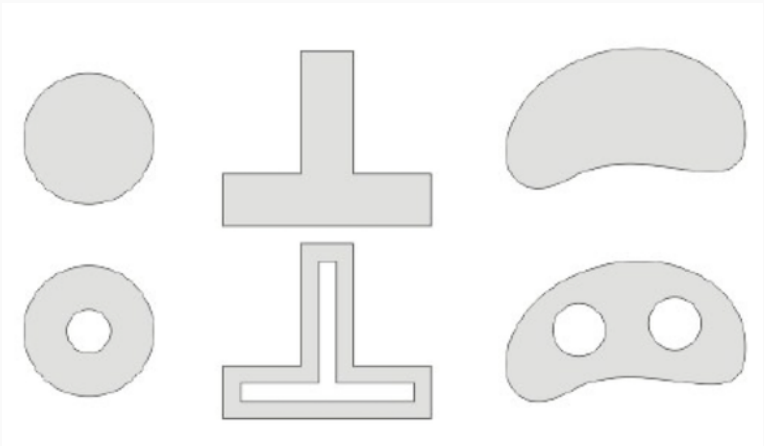
El **método de Gauss-Seidel** reemplaza
el valor del nuevo valor en la misma iteración.
Se tienen N^2 ecuaciones para N^2 incógnitas
Las operaciones se repiten hasta
la convergencia de la diferencia entre los valores
antiguos y los nuevos $diff = MAX|u_{i,j}^{k+1}, u_{i,j}^{k}|$
$diff \leq \epsilon$

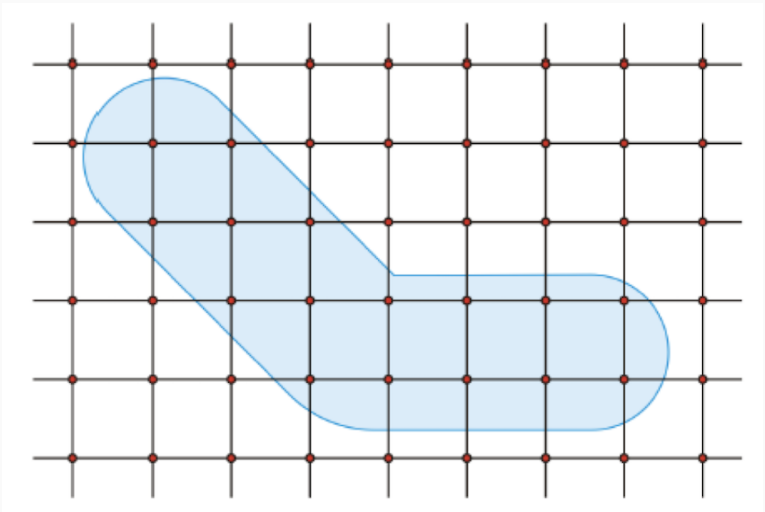
Complejidad: T(n)= kmN^2

donde N son los nodos en cada dimensión, m las operaciones por nodo y k el número de iteraciones

Descomposición de dominio

La solución de EDP implica una malla numérica para definir el dominio. Normalmente se divide en subdominios y se asigna a cada proceso la solución de un subdominio. Pero la mayoria de sistemas no son rectangulares (hay casos en los que se puede encerrar el dominio en rectángulos, mayor consumo de memoria)

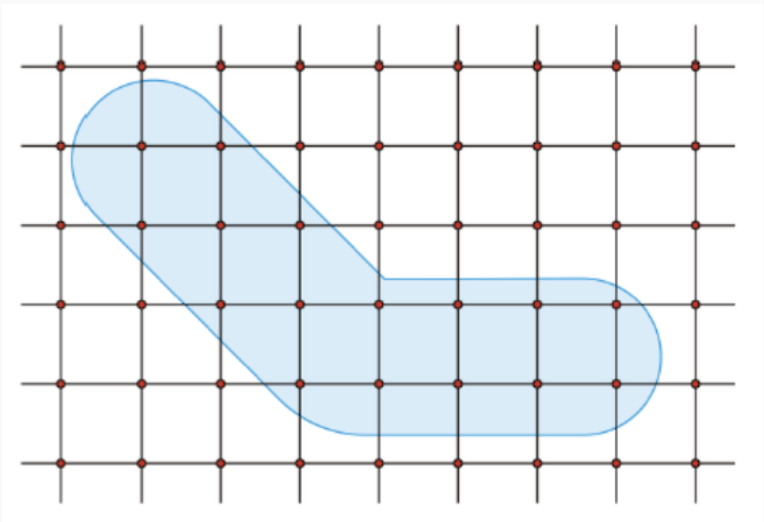


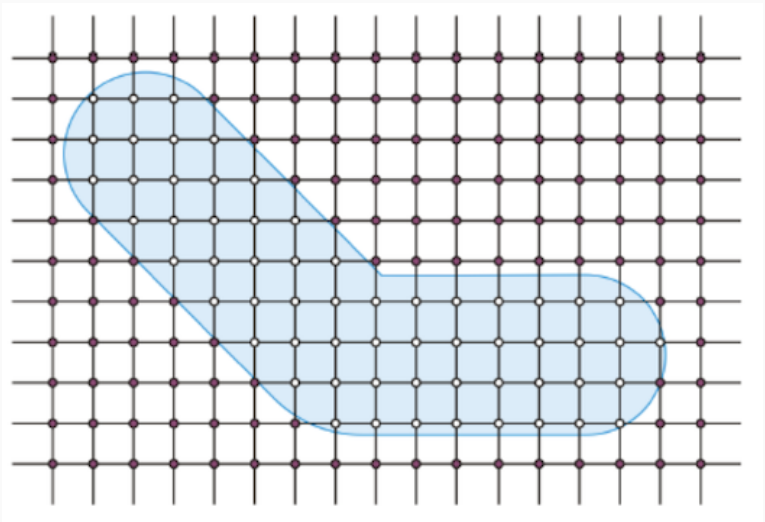


Un dominio irregular puede ser descompuesto, y los puntos fuera del dominio deben ser conocidos

Descomposición de dominio

Incrementar el número de puntos es menos eficiente

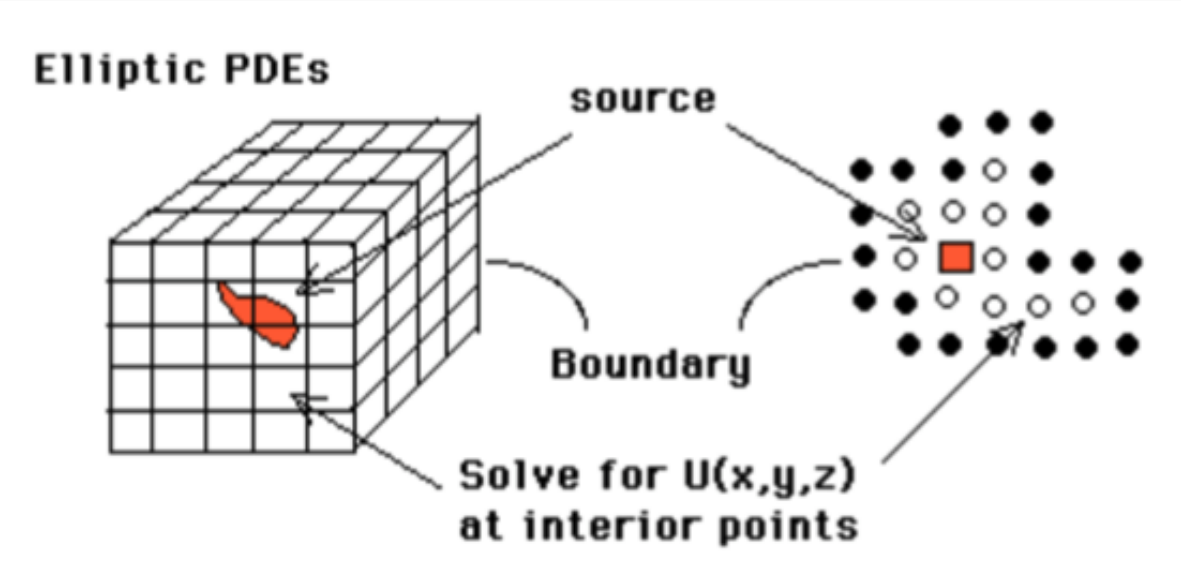




Existirán algunas simetrías, pero no en todas las áreas del dominio

Descomposición de dominio

Si el dominio es cúbico se puede descomponer fácilmente, pero es más difícil si tiene forma arbitraria, ya que para lograr **load balancing** se necesita igual número de puntos de malla en cada proceso



Descomposición de dominio

E.g. la descomposición de dominio en un modelo PIC (Particle in Cell) para simulación de plasmas. Código Smilei (Deroullat et.al. 2017)

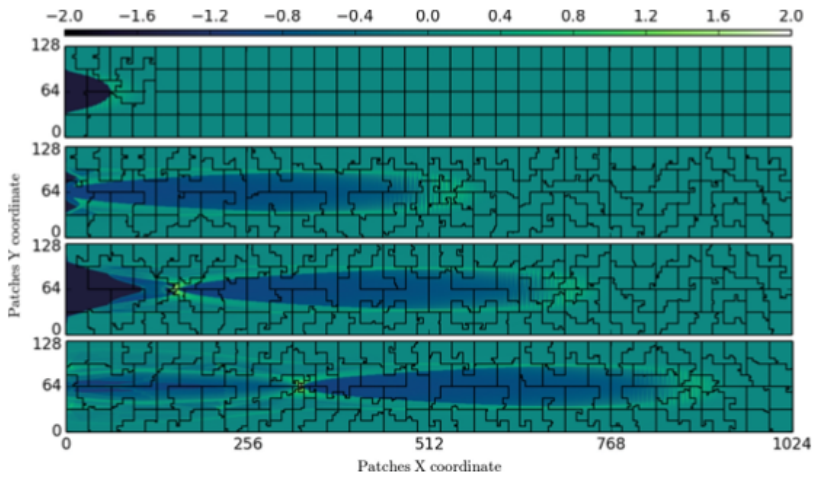


Figure 6: Evolution of the MPI domains shapes with respect to the computational load distribution. The colormap indicates the local imbalance $I_{loc} = \log_{10}(L_{loc}/L_{av})$ where L_{loc} is the local patch computational load and L_{av} the average computational load. Black lines delimit the different MPI domains. The laser enters an initially homogeneous plasma from the left side of the box and propagates towards the right. The 4 panels show the entire simulation domain after 1600, 5480, 6820, 9080 iterations from top to bottom.

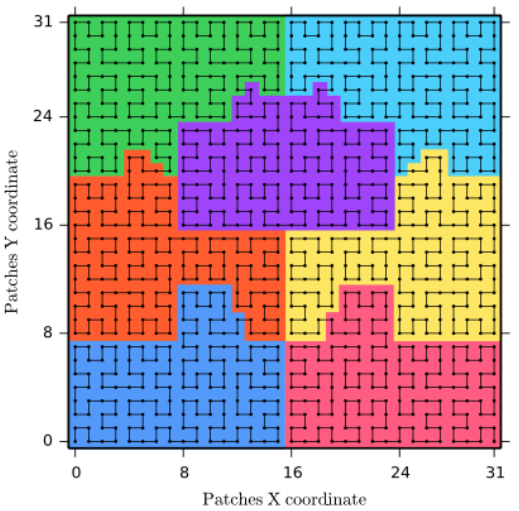


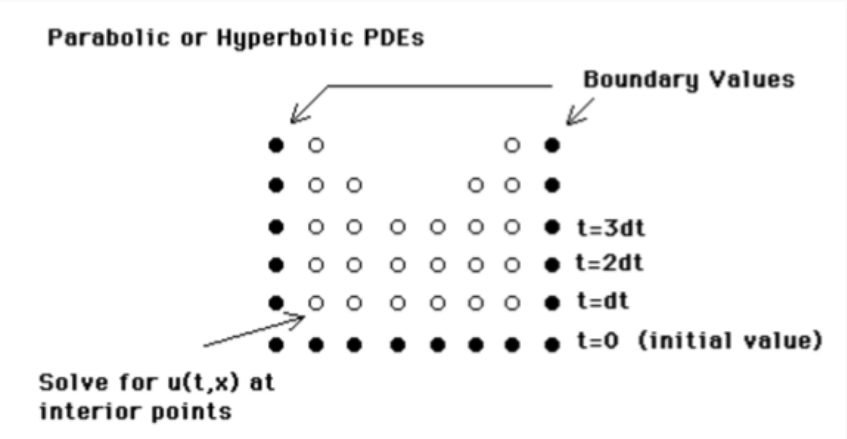
Figure 3: Example of a 32×32 patches domain decomposition, shared between 7 MPI processes. MPI domains are delimited by different colors. The Hilbert curve (black line) passes through all the patch centers (black dots). It starts from the patch with coordinates $(0, 0)$ and ends at the patch with coordinates $(31, 0)$.

Ecuación Parabólica (de difusión)

$$\frac{\partial u}{\partial t} = -D\frac{\partial}{\partial x}(\frac{\partial u}{\partial x})$$

donde

D es un coeficiente de difusión.



La solución es el valor del campo u(x,t) a un tiempo t dadas las condiciones iniciales en t=0, y sujeto a determinadas condiciones de frontera

La descomposición del dominio se aplica al espacio, ya que el tiempo debe ser secuencial, y no puede ser paralelizado

Ecuación Parabólica en paralelo

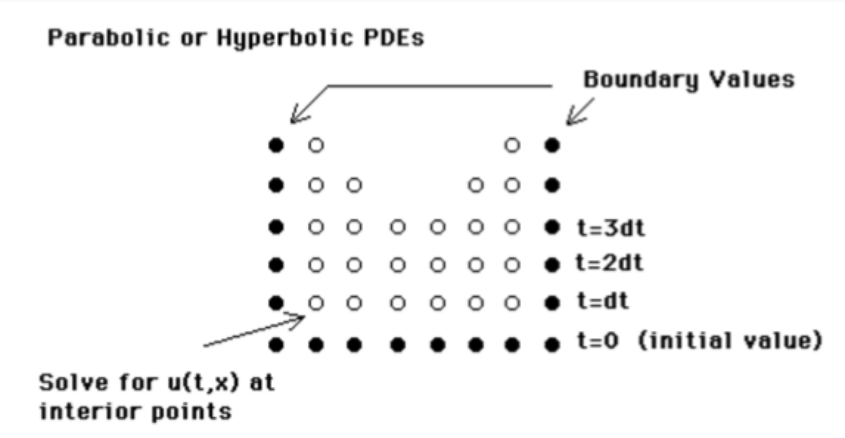
Se hace asignando un subdominio (o máximo, un número pequeño de subdominios) a cada proceso. Para ello:
- Se debe ejecutar igual trabajo en cada subdominio (**load balancing**)
- Se debe minimizar comunicación **node to node** (MPI_Send, MPI_Recv)

Normalmente, condiciones geométricas o físicas del dominio son preponderantes

Para la discretización usada:

$$\frac{\partial u}{\partial x} = \frac{u_{i+1} - u_{i-1}}{2h}$$

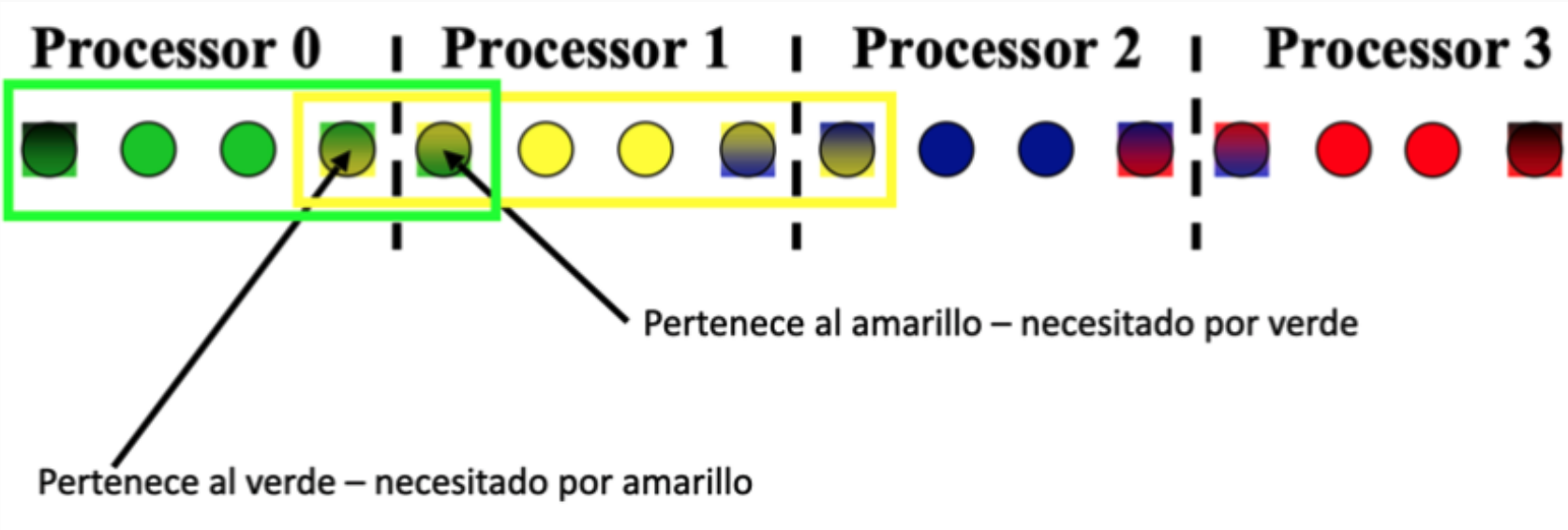
$$\frac{\partial^2 u}{\partial x^2} = \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2}$$



Para calcular derivadas de las fronteras de los subdominios se requieren los valores de los subdominios adyacentes, que están a cargo de otros procesos, lo que requiere comunicación entre ellos

Ecuación Parabólica en paralelo

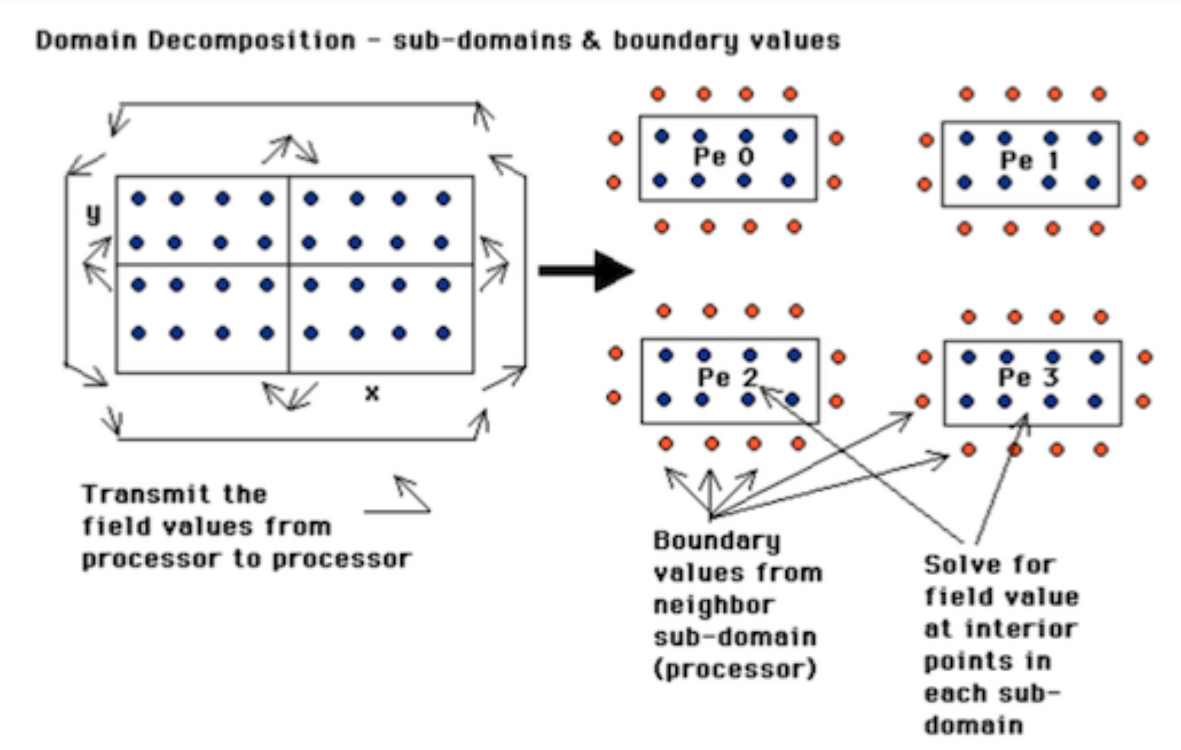
El problema de comunicación Node-to-node en una dimensión



Se necesita comunicación P2P para intercambiar puntos de frontera vecinos

Ecuación Parabólica en paralelo

El problema de comunicación Node-to-node en dos dimensiones



Ecuación hiperbólica de onda

Una cadena oscilante se describe con una ecuación de onda

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$

que representa un sistema que evoluciona en el tiempo



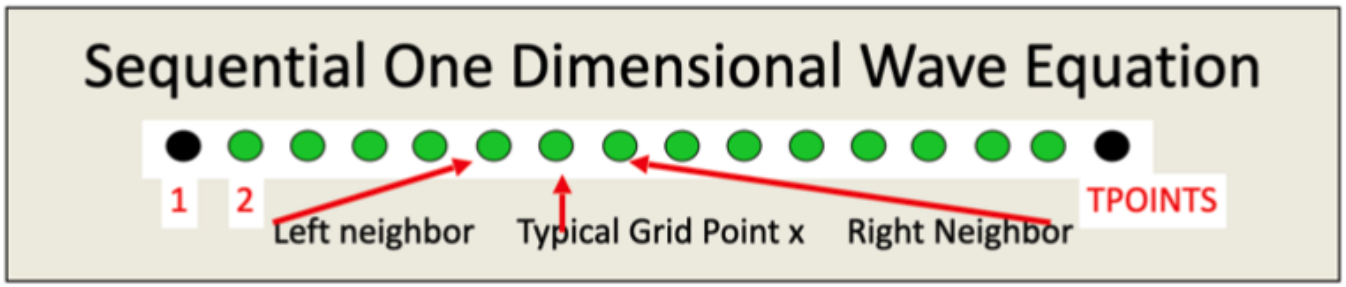
Ecuación hiperbólica de onda

Discretizando

$$\frac{(u(x,t+1)-2u(x,t)+u(x,t-1))}{\delta t^2} = c^2 \frac{(u(x+1,t)-2u(x,t)+u(x-1,t))}{\delta x^2}$$

O también, para calcular u en un paso en el futuro, basado en x actual y un paso en el pasado

$$u(x,t+1) =$$
$$2u(x,t) - u(x,t-1) + (u(x+1,t) - 2u(x,t) + u(x-1,t))(c^2\delta t^2/\delta x^2)$$



Bibliografía i


David B. Kirk and Wen-mei W. Hwu *Programming Massively Parallel Processors: A Hands-on Approach*. 2nd. Morgan Kaufmann, 2013. isbn: 978-0-12-415992-1.


Norm Matloff. *Programming on Parallel Machines*. University of California, Davis, 2014.


Peter S. Pacheco. *An Introduction to Parallel Programming.* 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5.


Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP.* 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.


Jason Sanders and Edward Kandrot. *CUDA by Example: An Introduction to General-Purpose GPU Program- ming.* 1st. Addison-Wesley Professional, 2010. isbn: 0131387685, 9780131387683.

UTEC
UNIVERSIDAD DE INGENIERÍA
Y TECNOLOGÍA

